Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

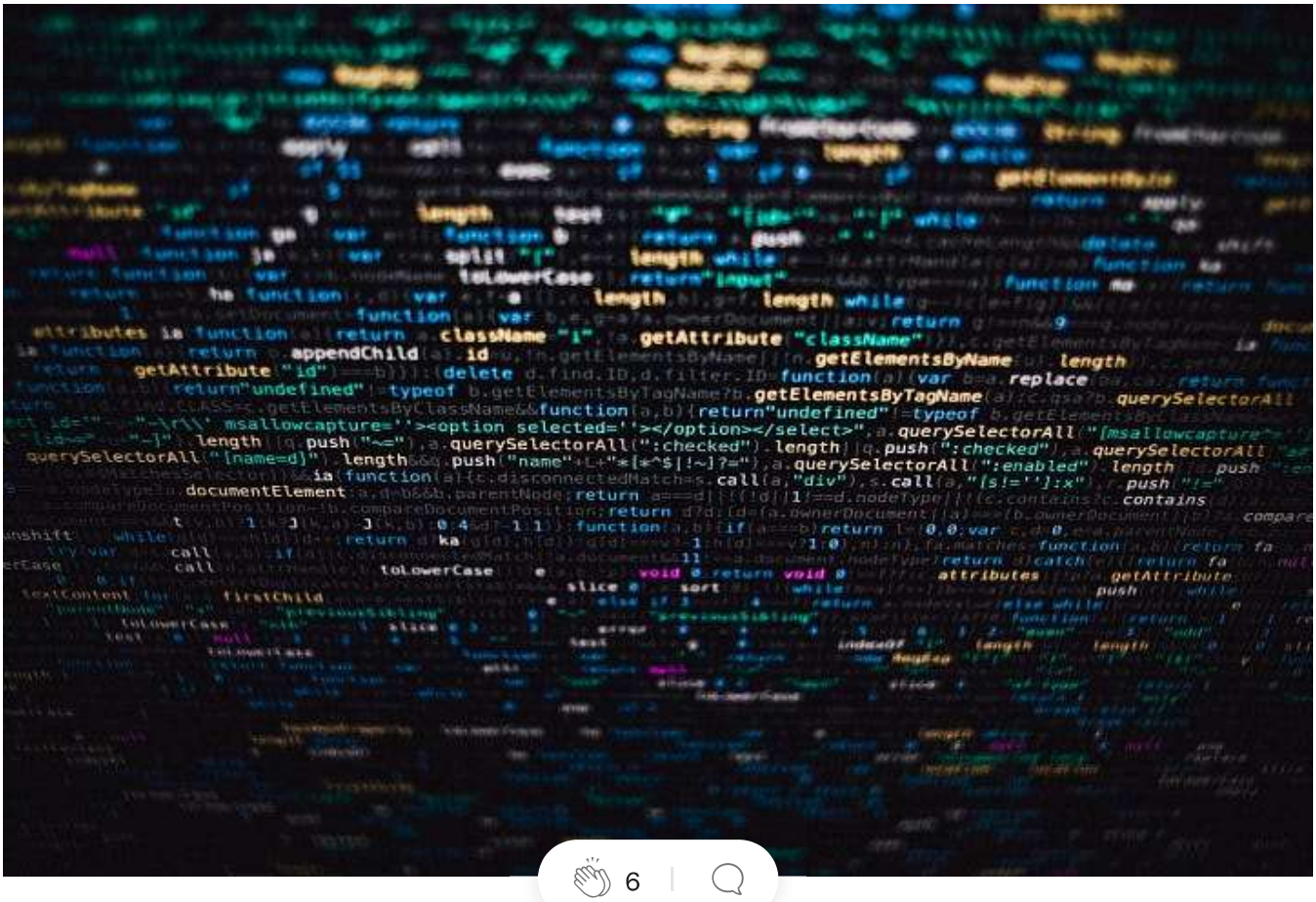
Sep 23, 2020 · 6 min read · ✨ · Listen

Save

SPACE SCIENCE WITH PYTHON

Asteroid Project (Part 2) — Test Driven Development

Part 22 of the tutorial series continues with the second part of our science project. Before we dive deeper into Python and Near-Earth Objects, let's take a look at Test Driven Development



6

Photo by Markus Spiske on Unsplash

Preface

This is the 22nd part of my Python tutorial series "Space Science with Python". All codes that are shown in the tutorial sessions are uploaded on GitHub. Enjoy!

Introduction

No, we are not yet starting with any Near-Earth Object (NEO) related Python development or implementation. We need to sharpen the axe before we can cut any tree. In our case: Let's dive into some development concepts that will lead to a sustainable long-term project and Python library.

How do you code? Most developers (either free-time coders or professionals) like to see quick results; they develop a prototype, some clickable interface, or demo page to achieve a first product milestone. In a professional working environment this may also be caused by external factors like deadlines, product owners, business partners or customers that are expecting results within a certain time frame. Some may talk past each other, caused by different professional backgrounds and knowledge.

The result? For example: Spaghetti Code that is not tested well enough and that is difficult to maintain, to upgrade and to understand!

In our science project we want to develop a reliable and sustainable Python solution. Something that can be used by amateur or professional astronomers working on NEOs. And that can also be used by developers and coders to create new functionalities that can be merged with the library.

But how can we ensure (from the beginning on) the sustainability and maintainability of our upcoming library? Well, besides a proper project plan and program structure (PEP8 formatting, PEP257 and Numpy Docstrings) there is one concept from the world of agile working that appears to be tedious, somehow boring … : Test Driven Development (TDD).

Test Driven Development

TDD reverses the “classical programming approach”. The first-coding-then-testing paradigm is changed to **first-testing-then-coding**. The rules appear to be simple, yet confusing:

1. **Write a single (unit) test for a class / function / …**
2. **Write production code to succeed the failed unit test**
3. **Do not continue with other functionalities until all unit tests succeed**

These are harsh rules. First, I need to define a unit test for a non-existing function. This test fails obviously. Now I develop some code to succeed the unit test. When the test passed another unit test is added, eventually for the same function again to prove its robustness or generic implementation. Did it succeed again, and again and again? Good! Your function appears to be suitable to be used later. If not, re-code it until all tests pass.

If all unit test for a single class or function pass define and write new unit tests for the next element of your coding project.

Now this approach may sound boring. One wants to achieve great things in a short time! Why bothering with this tedious approach? Where is the pioneering spirit? Well even if you work on a truly innovative product or idea, you will need basic functionalities; totally independent of what you want to achieve. *Testing-First* ensures that your work will have a solid fundament to rely on. In the long run, a code with several initial bugs is more time consuming to maintain and e.g., if your scientific work relies on it … well … in the worst case you have to withdraw your analysis or numerical simulations.

Modes of TDD

Photo by Kevin Ku on Unsplash

In theory, TDD has three working / coding modes (these modes easily merge into another). Let's assume you created a unit test, based on the product's requirements, and you need to develop the corresponding product code:

- *Obvious Implementation*

The *Obvious Implementation* is the most trivial mode. Based on a unit test example you can clearly derive a generic function to solve a problem. Example: numerical values that are stored as a string need to be converted to an actual float. You immediately now that '23.15' can be converted to 23.15 by applying float('23.15').

A unit test needs to pass! You have no clue how to do it, so you simply fake the result! You set a few constants and return the hard-coded answer the assertion test is expecting. Of course you *Fake it* (the result). However, based on the faked constant you try to develop backwards to the actual input. Eventually, after some thinking and research you will find the *Obvious Implementation* that was not obvious at the beginning.

- *Triangulation*

The *Fake it* approach assumes that based on faking constants and a single test the developer will eventually determine a generic solution for the unit test problem. However, if a problem is too complex, more tests are needed that cover more cases. More and more hard coded solutions help one to *triangulate* the actual generic solution of a particular problem. Assume you have the task to download a file from a website. The unit test calls a function that requires to return a message like "Download succeeded", however you do not know where to begin with it. At this point you can fake the outcome. Create a function that will contain the download functionality, create a string with the entry "Download succeeded" and return it. Afterwards another unit test is added e.g., a Hash-Value of the downloaded static file. Again, you fake the result again by faking the resulting hash value of a mock-up file. More and more tests are added that ask you to check the servers response and so on. With more tests and more required functionalities you develop a code that needs to fulfil all requirements in the long run. Faked solutions will shift to generic solutions.

Refactoring & Documentation

Finally, if the tests pass, consider to refactor your code. In Python, you shall follow the PEP8 standards that ensure a high level of code styling …

PEP 8 -- Style Guide for Python Code

The official home of the Python Programming Language

www.python.org

… provide comments to explain what your code is doing and provide a documentation in PEP257 style (and / or Numpy Doc style or other styles you find suitable):

PEP 257 -- Docstring Conventions

The aim of this PEP is to standardize the high-level structure of docstrings: what they should contain, and how to say…

www.python.org

numpydoc docstring guide - numpydoc v1.2.dev0 Manual

This document describes the syntax and best practices for docstrings used with the numpydoc extension for Sphinx. Note...

numpydoc.readthedocs.io

Refactoring and documenting your code will help you also to understand your own code. Maybe some parts contain logical errors that are not covered by unit tests? Maybe you missed an important feature? Coding, refactoring and documenting go hand in hand. Programming should not be a pure abstraction of a problem, but also human readable to ensure long-life maintainability and sustainability.

Additionally, profiling your functions may then help you to boost the performance of your product.

Conclusion & Outlook

Today we covered some basic principles of Test Driven Development, short TDD. TDD shall help us to develop a sustainable and Python library for our Space Science NEO project that provides tested and accurate results. Before we dive deeper into the scientific part of this project, we will go through a TDD example next time. There we will go step-by-step through the mentioned TDD process since during the project you will only see the results and successful unit tests of some coding sessions (maybe an additional video blog would be useful for some interested readers?).

If you struggle with coding, if you are lost in your own development process, objects and functions, remember to read the *Zen of Python*:

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren’t special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one — and preferably only one — obvious way to do
it.
Although that way may not be obvious at first unless you’re Dutch.
```

```
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it’s a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea — let’s do more of those!
```

Thomas

Literature & References

Test-Driven Development - An Empirical Evaluation of Agile Practice | Lech Madeyski | Springer

Agile methods are gaining more and more interest both in industry and in research. Many industries are transforming...

www.springer.com

Python Software Development Agile Data Science

Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don’t already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play